Trainer Guide: JDBC - Hands-on Lab

A walk-through script for jdbc-lab.html - building full database CRUD from a first connection to a clean DAO. Explain every step out loud as you lead.

Presenter: Purvam Prajapati - Faculty, AHD
Companion to: jdbc-lab.html

Each lab has an ON SCREEN reminder and a SAY / WALK THROUGH section you can read out as you lead. EMPHASISE and DELIVERY TIP notes are for you only.

Setup

Lab flow, schema box & the H2 fallback

ON SCREEN

The lab page: a pinned employees-table schema box, an 8-step checklist, and lab sections with copy buttons. An H2 alternative box in the pre-requisites.

SAY / WALK THROUGH

Today we go from 'my data disappears on restart' to a complete, persistent CRUD application, and then refactor it into a clean DAO. The employees-table schema is pinned at the top - every lab uses that one table, so keep it in view. One safety valve to mention up front: if Derby setup gives anyone trouble, there's an H2 fallback box. H2 is a tiny zero-setup database - change just three things, the URL, the driver class, and the dependency, and every line of lab code works identically. Keep that in your back pocket in case Derby fights someone.

Pre-requisites Check

ON SCREEN

Pre-requisites: JDK 17+, the Derby JAR on the build path, and a project with a com.tcs.jdbc package.

SAY / WALK THROUGH

Pre-requisites: JDK 17 or higher - check with java dash version. The Derby JAR on the build path - in Eclipse it's bundled, or download Apache Derby. And a Java project ready with a package com dot tcs dot jdbc, where we'll create all our classes. Tick each as you confirm it. If the Derby JAR isn't found later, the fix is Project, Build Path, Add External JARs - point that out now so it's not a surprise.

The labs - build database CRUD

Lab 0 - Derby Setup in Eclipse

ON SCREEN

Steps in the Data Source Explorer to add a Derby connection, the JAR, test it, and copy the JDBC URL. Expected URL with create=true.

SAY / WALK THROUGH

Lab zero - we get a working connection before writing any Java, using Eclipse's Data Source Explorer so we can test it visually. Window, Show View, Other, Data Management, Data Source Explorer. Right-click Database Connections, New, choose Derby. Add the derby JAR. Set the database name to employeedb, user sa, password blank. Click Test Connection - it should succeed - then copy the generated JDBC URL. The URL looks like jdbc colon derby colon path slash employeedb semicolon create equals true. Explain that create equals true tells Derby to create the database if it doesn't exist - essential for the first run. Without it, the very first connection fails because the database isn't there yet.

Lab 1 - Your First JDBC Connection

ON SCREEN

A JDBCTest class running all five steps - register, connect, verify, close - with expected output 'Connected: true / Closed: true'.

SAY / WALK THROUGH

Lab one - all five JDBC steps in their simplest form, the tactile anchor before CRUD. Create the package com dot tcs dot jdbc and a class JDBCTest. The code does five things: Class dot forName registers the driver - optional in modern JDBC but shown for clarity; DriverManager dot getConnection opens the connection with the URL, sa, and blank password; we print not-con-dot-isClosed to verify it's open; and con dot close releases it. Run it. You're looking for two lines: Connected true, then Closed true. That's the whole JDBC lifecycle in one tiny program. If you get a ClassNotFoundException, the Derby JAR isn't on the classpath - back to Lab 0.

Lab 2 - Create Table + INSERT

ON SCREEN

Part A: a CREATE TABLE statement via a Statement. Part B: an insertEmployee method using a PreparedStatement with setInt/setString, called three times.

SAY / WALK THROUGH

Lab two has two parts. Part A creates the table - a CREATE TABLE statement run with a Statement and executeUpdate. Mention that running it twice throws 'table already exists' - that's fine, either wrap it in a try-catch or drop the table first. Then verify in Data Source Explorer that the EMPLOYEES table appears. Part B is the important one - INSERT with a PreparedStatement. Write an insertEmployee method: the SQL has five question-mark placeholders, then setInt and setString bind each value - and stress that the parameter index starts at one, not zero. Call it three times with different employees, and you'll see three 'one row inserted' lines.

EMPHASISE

Do the deliberate experiment: insert emp_id 101 twice and read the error. Point out the SQLState code 23505 - duplicate primary key. It both teaches SQLState and sets up the exception-handling lab coming in Lab 5.

Lab 3 - SELECT All - Reading a ResultSet

ON SCREEN

A SELECT star query with a while(rs.next()) loop printing a formatted table of all employees.

SAY / WALK THROUGH

Lab three - reading data back. Run SELECT star FROM employees with executeQuery, which returns a ResultSet - a cursor over the rows. Then a while loop on rs dot next, pulling each column with rs dot getInt and rs dot getString, printed as a neat table. Explain how the cursor works, because it confuses beginners: rs dot next starts before the first row; each call moves forward one row and returns true while rows remain, false when they're exhausted - so the while loop ends naturally. You read columns by name, like getString of first_name, or by 1-based index.

DELIVERY TIP

If there's time, do the 'try it yourself' - a getEmployeesByDepartment method using WHERE department equals question-mark with a PreparedStatement. It reinforces parameter binding in a SELECT, not just an INSERT.

Lab 4 - SELECT by ID, UPDATE, DELETE

ON SCREEN

SELECT by id using if(rs.next()) for found-vs-not-found; an UPDATE and a DELETE, each checking rows-affected.

SAY / WALK THROUGH

Lab four completes CRUD. SELECT by id uses if-rs-dot-next rather than a while loop, because we expect at most one row - and it gives a clean not-found branch. Test it with id 101, which exists, and id 999, which doesn't, so they see both paths. Then UPDATE - set a new salary where emp_id matches - and DELETE - remove where emp_id matches. Both use executeUpdate, which returns the number of rows affected. Check rows greater than zero to print 'updated' or 'employee not found'. Run SELECT all after each operation so they watch the change land.

EMPHASISE

Highlight why the rows-affected check matters: executeUpdate returning zero means 'ran fine but matched nothing', versus a positive number meaning 'found it and changed it'. That distinction is far more useful than just assuming success.

Lab 5 - try-with-resources Refactor

ON SCREEN

The insert method refactored into try-with-resources, with a catch that prints message, SQLState, and error code. A deliberate duplicate-key failure.

SAY / WALK THROUGH

Lab five makes the code production-safe. Every lab so far has a hidden risk: if an exception fires between getConnection and con dot close, the connection leaks - and enough leaks exhaust the database. The fix is try-with-resources: declare the Connection and PreparedStatement in the try parentheses, and Java closes them automatically, on success and on exception. Refactor insertEmployee this way, and in the catch block print all three pieces of SQLException info - getMessage, getSQLState, getErrorCode. Then deliberately trigger a failure by inserting a duplicate emp_id, and watch those three print: a message, SQLState 23505, and an error code. That's exactly the information your DevOps team needs to diagnose a production database issue.

DELIVERY TIP

Set the refactor task: have them update their SELECT, UPDATE, and DELETE methods to try-with-resources too - and for SELECT, declare the ResultSet in the try header as well, since it's AutoCloseable.

Lab 6 - DAO Pattern - Full Refactor

ON SCREEN

Three classes: DBConnector (just the connection), EmployeeDAO (all SQL methods), and EmployeeTester (a Scanner menu). The biggest block in the lab.

SAY / WALK THROUGH

Lab six pulls it all together into the DAO pattern - three classes, three responsibilities. DBConnector has one job: a getConnection method, no SQL, no UI. EmployeeDAO holds all the SQL operations - insert, getAll, getById, update, delete - each using DBConnector and try-with-resources. And EmployeeTester is a Scanner menu with a switch: add, view, search, update, delete, exit - looping until the user enters zero. Run EmployeeTester and let them interact with their own database through the menu. This is the biggest block, so lean on the copy buttons.

EMPHASISE

End with the observation question and let them answer it: if you switched this project from Derby to MySQL, which files change? Only DBConnector - its URL and driver, and maybe minor SQL. EmployeeTester is untouched. Then land it: Spring Data's JpaRepository IS a DAO - when you call repository dot findById, it runs exactly this JDBC, generated for you.

Reference & close

Complete code reference & wrap-up

ON SCREEN

The complete code reference with the H2 alternative config, and the quick-reference card with interfaces, the five steps, and SQLState codes.

SAY / WALK THROUGH

The reference at the bottom has the H2 alternative config - the three things to change if Derby didn't cooperate - and a reminder about run order on later runs, since the table already exists. The quick-reference card lists the core interfaces, the five steps, and the common SQLState codes. To wrap: today you went from data that vanished on restart to a complete, persistent CRUD application, with safe PreparedStatements, real exception handling, and a clean DAO architecture. And the big connection - everything Spring Data JPA does is built on exactly this. When you later call repository dot save, the JDBC you wrote today is what runs underneath, generated for you. Now you know what the magic actually is. Great work.